

CITY-G: Publisher-Blind, Time-Blind End-to-End Encryption for Massive Asynchronous Groups

City-G Working Group

Version 0.1.1

February 11, 2026

Abstract

Large-group end-to-end encrypted messaging is difficult to scale without giving up either security guarantees or operational simplicity. Existing approaches often force strict global ordering for key schedule evolution, rely on wall-clock assumptions, or expose revocation-side metadata that degrades privacy at scale. This paper presents CITY-G, a protocol profile designed for high-concurrency, asynchronous environments with millions of members and strict server-side blindness over message secrets.

CITY-G combines a time-blind forward-secrecy schedule, deterministic acceptance, and a post-revocation secrecy (PRS) barrier based on a KEM-tree cover. The design separates public acceptance state from client secret state, enforces canonical wire encoding and raw-byte commitment discipline, and introduces explicit fail-closed rules for malformed or adversarial updates. We describe the protocol architecture, state machine, and security properties; analyze concurrency and scalability behavior; and compare CITY-G with mainstream group key management patterns. The result is a protocol that keeps steady-state operations lightweight while making revocation handling explicit, verifiable, and robust under active-server threat assumptions.

1 Introduction

End-to-end encrypted (E2EE) group systems must simultaneously satisfy three pressures: security under active attackers, asynchronous operability for offline devices, and production-grade scalability under bursty concurrency. In large deployments, those pressures collide. Systems that optimize for simplicity often centralize trust in servers for key distribution. Systems that optimize for strict cryptographic structure frequently impose serialization points that become throughput bottlenecks. Systems that optimize for throughput may under-specify edge-case validation, enabling interop drift or latent security regressions.

CITY-G targets this gap: robust security semantics without a server-side message secret role, explicit acceptance determinism without wall-clock dependence, and practical behavior for high-concurrency populations. The profile analyzed here is the final unified specification v0.1.1 (FS-hybrid + PRS barrier), in which forward secrecy, acceptance rules, and post-revocation secrecy are specified in a single normative document [3].

Core thesis. CITY-G is useful because it treats security invariants as state-machine invariants, not just crypto primitive selections. In practice, compromise and drift happen at boundaries: encoding,

ordering, replay, and recovery behavior. CITY-Gcodifies those boundaries with explicit MUST-level rules and verifiable transitions.

2 Problem Setting and Design Goals

2.1 Operational setting

We consider very large asynchronous groups where:

- clients may be offline for long intervals,
- network delivery is adversarial and reordering is normal,
- joins and regular sends occur concurrently at high rates,
- revocations are infrequent but high impact,
- acceptance infrastructure must scale horizontally.

2.2 Threat model

The adversary may control transport and publishers, reorder or replay traffic, and attempt structural malleation. Excluded or revoked members may collude with compromised devices. Store-now-decrypt-later adversaries are in scope. The acceptance server is not trusted with message secrets and is treated as potentially curious; selected checks also model an active malicious server.

2.3 Goals

1. **Confidentiality without server message secrets:** server cannot decrypt payloads or header-protected message material.
2. **Time-blind forward secrecy:** acceptance must not rely on wall clocks.
3. **Post-revocation secrecy:** once revocation roots change, barrier advancement is required before normal traffic resumes.
4. **Deterministic acceptance:** canonical encoding, closed-world headers, bounded maxima, and fail-closed checks.
5. **Concurrency at scale:** preserve high steady-state throughput for non-revocation anchors.

3 System Model and State Separation

3.1 Roles

- **Clients:** hold long-lived and evolving secrets, produce anchors, encrypt/decrypt payloads.
- **Acceptance server:** validates anchors and manages public state transitions.
- **Membership/SRX subsystem:** provides revocation and join enumerations with integrity.

3.2 Anchor types

CITY-G defines three anchor types: **JOIN**, **REGULAR**, and **MERGE**. Barrier update material (key 175) and merge-only keys force **MERGE** typing; join leaf key publication (key 177) forces **JOIN**. The profile uses a closed-world key registry with strict mode-specific presence rules [3].

3.3 State separation

A critical design choice is separating:

- **Public state** (server): acceptance counters, per-device chain cursors, barrier public tree commitment, revocation hash commitment.
- **Secret state** (client): FS chain secret, epoch derivations, barrier key, private KEM keys on self path.

This separation prevents accidental “server helpfulness” from becoming a secrecy dependency.

4 Cryptographic and Encoding Discipline

4.1 Primitive suite

The profile uses BLAKE3-based domain-separated hashing, HKDF-BLAKE3 with fixed output length 32, ChaCha20-Poly1305 AEAD, and ML-KEM-768 for cover operations. Deterministic CBOR (RFC 8949 profile) is mandatory for all critical structures [2, 4, 5].

4.2 Why canonical encoding is first-class

In distributed cryptographic systems, parser-level ambiguity is often as dangerous as primitive breakage. CITY-G therefore requires deterministic re-encoding checks and maps failures to explicit codes. It also distinguishes semantic objects from their raw transmitted bytes where identity is digest-bound (e.g., barrier update bytes).

5 Protocol Overview

5.1 FS-hybrid schedule

Each anchor carries a monotone epoch counter **fs_ec**. Clients evolve local FS chain secret **K_fs** and derive per-epoch material (including **tau_e**) via profile-fixed labels. The profile binds device-level chain commitments to barrier state by defining:

$$\begin{aligned} \text{fs_dev_commit} = H_L(\text{"fs/dev/chain/v2",} \\ [\text{header}[108], \text{header}[141], \text{header}[152], \\ \text{header}[176], \text{barrier_update_digest}]). \end{aligned} \tag{1}$$

This construction ties device progression to barrier progression.

5.2 Payload envelope and nonce safety

Payload ciphertexts are carried in a 3-field envelope with explicit `msg_index`. Message key and nonce derivation include `msg_index`; uniqueness is mandatory for each fixed identifier tuple `{gid, weid, t, xk_hash, E_k, barrier_version}`. State enforcing uniqueness must be crash-safe.

5.3 Time-blind acceptance and FLG

Acceptance never uses wall-clock checks. Instead, forward-leap guards (FLG) bound `fs_ecjumps` against persisted acceptance state and policy-derived windows. This is a key scaling and reliability property: skewed clocks cannot silently widen exposure windows.

5.4 PRS barrier overview

The barrier is a committed public KEM tree with client-held private keys along self paths. Revocation updates are represented by barrier updates that include:

- pre- and post-update tree commitments,
- deterministic new internal public keys on updater path ancestors,
- node ciphertexts that transport ancestor path secrets to copath resolutions.

Clients recover a unique matching ciphertext, derive upward secrets, compute new `K_barrier`, and update local key material.

5.5 Updater and recover semantics

The updater is explicitly excluded from running the generic recover path for its own update. Instead, it must persist pending activation state before publish, correlate accepted bytes by digest, then activate. This avoids false-local failures and crash-induced divergence.

Acceptance skeleton for a barrier-carrying merge.

1. Parse deterministic header map; reject unknown keys or invalid maxima.
2. Enforce anchor-type presence matrix and closed-world policy.
3. Verify FS base, device-chain commit, and FLG bounds.
4. Parse and deterministic-check `BarrierUpdate` and `KemTreeCoverPayload`.
5. Validate path structure, canonical ordering, and exact `ExpectedNodeSet`.
6. Build `snapshot_pre`; verify `kem_tree_hash_before`.
7. Apply `CP.new_public_keys`; verify `kem_tree_hash_after`.
8. Validate `ExpectedPairs` completeness/minimality.
9. Verify proofs in fail-fast order; then atomically commit public state.

Table 1: Asymptotic cost profile (high-level)

Operation	Dominant work	Notes
Join/regular acceptance	Header/proof checks, $O(1)$ state writes	Fast path when revocation roots unchanged
Barrier merge acceptance	Tree reconstruction + hash-chain + ExpectedPairs	Cold path; cost scales with change set and coverage
Client recover (non-updater)	Match scan + one successful decap + path derivation	Fail-closed on 0 or multi-match
Updater rekey	Path secret derivation + KEM encaps per target pair	Burst cost concentrated on revocation events

6 Concurrency and Scalability Behavior

6.1 Steady-state path

Under stable revocation roots, `JOIN` and `REGULAR` anchors do not require barrier update bytes and do not bump barrier version. This keeps high-frequency traffic on a lightweight path while preserving deterministic acceptance.

6.2 Revocation synchronization point

When revocation roots change, `CITY-Gin` introduces an explicit synchronization requirement: the next admissible transition must be a merge carrying barrier update and barrier version increment. This is intentional. It localizes revocation cost to a cold path and prevents accidental post-revocation plaintext recoverability under stale barrier keys.

6.3 Large-group complexity

The design choice is clear: optimize daily throughput; pay structured complexity only when revocation semantics demand it.

7 Security Analysis

7.1 Property matrix

7.2 Active-server resistance

Server-side validation is necessary but not sufficient against a malicious server controlling public-tree responses. `CITY-G` addresses this by requiring updater and full clients to authenticate fetched public tree snapshots against previously committed hashes before proceeding. Snapshot auth failures are explicit diagnostics (960.9).

7.3 Why the barrier gate matters

Without mandatory barrier advancement on revocation-root change, senders could continue deriving payload keys under stale barrier context. The profile now forbids that path by acceptance gating.

Table 2: Security and protocol properties in the final profile

Property	Status	Mechanism
Confidentiality	Achieved	End-to-end payload keys from client-held <code>tau_e</code> and <code>K_barrier</code> ; server keyless for message decrypt path
Authentication and integrity	Achieved	Anchor authentication + deterministic validation + proof commitments
Forward secrecy (time-blind)	Achieved	Evolving FS chain, zeroization guidance, no wall-clock acceptance dependency
Post-revocation secrecy	Achieved (policy-enforced)	Mandatory barrier update gating when revocation roots change
Membership consistency	Achieved	Commitment chain, deterministic tree/hash checks, ExpectedPairs rules
Asynchronicity and scalability	Achieved	Fast path for non-revocation anchors; explicit cold path for revocations
Concurrency	Achieved with bounded retries	Deterministic acceptance and barrier version gating; retries on races are explicit
Post-compromise recovery	Conditional	Requires compromise to cease and subsequent correct evolution/update
Deniability	Out of scope in this profile	Strong sender authentication is explicit design choice

This is the main reason post-revocation secrecy is a protocol property rather than an application convention.

8 Comparison with Related Approaches

8.1 Compared with MLS-style trees

Messaging Layer Security (MLS) standardizes tree-based group key management with strong formal structure [1]. CITY-G differs in emphasis:

- explicit time-blind acceptance and FLG-based epoch control,
- explicit server-keyless message path and raw-byte digest discipline,
- revocation-root to barrier-update gating as a mandatory acceptance condition,
- separate treatment of fast-path anchors and revocation cold path.

This is not a “better/worse” claim in the abstract; it is a design fit claim for high-concurrency asynchronous environments that prioritize deterministic acceptance behavior and operationally explicit revocation semantics.

8.2 Compared with sender-key fanout designs

Sender-key systems can provide excellent throughput but often centralize revocation complexity in server policy or out-of-band repair. CITY-Gcodifies revocation transition rules in the protocol state machine itself, reducing reliance on implicit operational assumptions.

9 Implementation Guidance

9.1 Non-negotiable engineering invariants

- Never re-encode digest-bound raw objects before identity checks.
- Treat deterministic encoding as a security property, not formatting hygiene.
- Persist uniqueness and anti-replay state crash-safely before use.
- Keep updater pending-state activation digest-correlated and atomic.
- Keep pairwise $(dk_t, pkhash_t)$ updates crash-safe.

9.2 Migration from FS-only profile

A practical migration plan should proceed in phases:

1. add header registry and state schema for barrier fields,
2. implement acceptance checks and error mapping in shadow mode,
3. ship barrier client code paths behind feature gates,
4. validate KATs and mixed offline/online recovery scenarios,
5. enforce revocation-change gating in production.

9.3 Test strategy

The profile’s mandatory KAT set should be treated as conformance gates, not documentation examples. In addition to vector tests, teams should run:

- parser differential tests for deterministic encoding rejection,
- fault-injection on updater crash/restart windows,
- adversarial tests for malformed ExpectedPairs and multi-match recovery.

10 Limitations and Future Work

- Barrier merges can become heavy under large, bursty revocation sets; operational budgeting remains important.
- Full verification depends on authenticated access to historical public tree states.

- Formal verification artifacts can be expanded beyond present conformance/KAT structure.
- Deniability is not a target of this profile due to explicit sender authentication requirements.

Future work includes deeper formalization of liveness under sustained adversarial churn, adaptive cover packing strategies, and reference interoperability suites across independent implementations.

11 Conclusion

CITY-G demonstrates that large-scale asynchronous E2EE can be both security-rigorous and operations-conscious when acceptance semantics are treated as first-class cryptographic boundaries. The protocol’s contribution is not a single primitive, but a coherent composition: time-blind FS progression, deterministic acceptance, explicit revocation-to-barrier gating, and client-verifiable cold-path recovery.

For deployments where revocation correctness, server blindness, and high concurrency are all non-negotiable, this profile offers a practical and auditable path forward.

A Normative Profile Reference

This whitepaper describes the final unified profile documented in:

`docs/specs.md` (CITY-G Unified Spec, v0.1.1 final).

The specification remains normative; this paper is explanatory.

B Notation Summary

Symbol / term	Meaning
<code>fs_ec</code>	Epoch counter carried on anchors
<code>tau_e</code>	Per-epoch FS-derived secret material
<code>K_barrier</code>	Barrier key bound to barrier version and revocation roots
<code>ExpectedNodeSet</code>	Required internal-node publication set on updater path ancestors
<code>ExpectedPairs</code>	Required ciphertext pair coverage from on-path source to copath resolution

References

- [1] Richard Barnes, Benjamin Beurdouche, Raphael Robert, Jon Millican, Christopher Patton, and Casper Wood. The Messaging Layer Security (MLS) Protocol. RFC 9420, RFC Editor, July 2023. URL <https://www.rfc-editor.org/rfc/rfc9420>.
- [2] Carsten Bormann and Paul Hoffman. Concise Binary Object Representation (CBOR). RFC 8949, RFC Editor, December 2020. URL <https://www.rfc-editor.org/rfc/rfc8949>.
- [3] City-G Working Group. CITY-G Unified Spec (FS-HYBRID + PRS BARRIER), v0.1.1, February 2026. Normative protocol specification in repository: `docs/specs.md`.

- [4] National Institute of Standards and Technology. Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203, U.S. Department of Commerce, August 2024. URL <https://doi.org/10.6028/NIST.FIPS.203>.
- [5] Yaron Nir and Adam Langley. ChaCha20 and Poly1305 for IETF Protocols. RFC 8439, RFC Editor, June 2018. URL <https://www.rfc-editor.org/rfc/rfc8439>.